

《深度强化学习与控制》第二周课程总结

邓一骏 25SD36037

May 2026

1、课程内容概述

本周讲了 MC 学习 (蒙特卡洛), TD 学习 (时序差分), SARSA 学习和 Q 学习, 同时引入了深度强化学习的概念。

本笔记作为课外学习资料记录, 首先是想从更便于理解的内容和例子来掌握贝尔曼方程, 价值函数, 动态规划。然后再对 MC 学习, TD 学习, SARSA 学习和 Q 学习进行进一步拆分。

1.1 贝尔曼方程的后向归纳法

从贝尔曼方程中可以得出, 我们可以从未来状态价值逐步推导到当前状态价值。这种“从后往前推”的计算过程, 在控制理论和强化学习中有一个经典的名字: 后向归纳法 (Backward Induction)。顺着的思路, 把这个“倒推”的过程用公式和图形具象化: 从终点到起点假设我们有一个非常简单的单向序列游戏, 一共只有 4 个状态: $s_0 \rightarrow s_1 \rightarrow s_2 \rightarrow s_{terminal}$ 。每个状态转移的即时奖励都是 $R = -1$ (比如希望尽快结束游戏, 走一步扣一分), 折扣因子 $\gamma = 1$ 。我们从后往前算:

1. 终点后面没有游戏了, 它的价值死死固定在:

$$V(s_{terminal}) = 0$$

2. 倒数第一步: 算出 $V(s_2)$ 现在我们站在 s_2 , 下一状态必定是 $s_{terminal}$ 。利用贝尔曼方程:

$$V(s_2) = R + \gamma V(s_{terminal}) = -1 + 1 \times 0 = -1$$

3. 倒数第二步: 算出 $V(s_1)$ 既然 $V(s_2)$ 已经被上一步“明码标价”成了 -1 , 我们站在 s_1 时, 根本不用管终点在哪, 直接看 s_2 即可:

$$V(s_1) = R + \gamma V(s_2) = -1 + 1 \times (-1) = -2$$

4. 倒数第三步: 算出 $V(s_0)$ 同理, 站在起点 s_0 , 直接利用已经算好的 $V(s_1)$:

$$V(s_0) = R + \gamma V(s_1) = -1 + 1 \times (-2) = -3$$

如果所有问题都是像上面这样单向、没有分支、不回头的长序列, 那我们确实只要从后往前扫一遍, 所有状态的价值就彻底解出来了。但在绝大多数现实问题 (比如控制系统、机器人导航、

下棋)中,会遇到两个麻烦:状态转移有“环”:比如在格子游戏里,机器人可以从 s_1 走到 s_2 ,但也可能因为噪声或控制指令又从 s_2 走回了 s_1 。这种情况下,根本找不到一个绝对的“倒数第一步”,它们互为因果。状态空间巨大:状态太多,或者我们一开始根本不知道完整的状态连接关系和转移概率 $p(s' | s)$ (也就是没有环境的模型)。

于是,强化学习出场了。正因为有上述麻烦,我们才不能只简单地“从后往前算一遍”。如果是已知模型的控制问题(动态规划):我们会给所有状态先随机猜一个初始价值(比如全设为0),然后把贝尔曼方程当成一个“刷新公式”,在这个网络里不断地循环迭代、互相传递价值,直到所有状态的价值都神奇地“收敛”稳定下来。如果是未知模型的纯强化学习(如 Q-learning, TD 算法):我们干脆不从后往前算了。我们让智能体在环境里从前往后真实地去跑、去试错。每走一步,就利用当前踩到的 s' 的估计价值,去微调 and 修正上一个状态 s 的价值。这就是所谓的“时序差分(Temporal Difference)”学习。

2、独木桥例子

加入环境转移概率,与策略的选择的独木桥例子

1. 场景设定(策略与环境完全剥离)

假设你踩在状态 $s_{\text{桥中}}$ (桥中央),需要过桥到达桥尾 s_1 ,过桥时可能因为环境/怪物追赶而坠落 $s_{\text{坠落}}$ 。这一次,你作为玩家,拥有真正的选择权,身后的怪物在逼近。你开始纠结是“稳扎稳打”还是“搏一把”,同时环境也充满了不确定性。

玩家的主观策略 π (策略选择概率):动作 $a_{\text{爬}}$:你的策略决定有 30% 的概率选择【爬】。动作 $a_{\text{跑}}$:你的策略决定有 70% 的概率选择【跑】。

大自然的物理规则 $p(s' | s, a)$ (环境转移概率)

一旦你做出了主观决定,大自然(环境)的物理规律开始结算:

1. 如果你选了 $a_{\text{爬}}$:动作太慢,有 60% 的概率被怪物追上并推落悬崖,转移到 $s_{\text{坠落}}$ 。运气较好,有 40% 的概率硬挺着爬到了桥尾,转移到 s_1 。

2. 如果你选了 $a_{\text{跑}}$:动作快,有 90% 的概率成功冲到桥尾,转移到 s_1 。脚滑,有 10% 的概率失足跌落悬崖,转移到 s 。

动作的即时回报 $r(s, a)$ (当下的代价/奖励)你在桥中央做出动作的瞬间,肉体和精神需要立刻付出代价或获得奖励(无论最终是否成功过桥):【爬】的即时回报 $r(s_{\text{桥中}}, a_{\text{爬}}) = -10$:双手双脚着地在粗糙的木头上磨砺,极其消耗体力,且耗时长导致心理压力极大,大自然立刻扣除 10 分。【跑】的即时回报 $r(s_{\text{桥中}}, a_{\text{跑}}) = -2$:直立冲锋,速度极快,动作潇洒且节省了宝贵的时间,大自然象征性地只扣除 2 分。

终点价值(边界条件):成功逃脱: $V(s_1) = +100$;坠落失败: $V(s_{\text{坠落}}) = -100$ (假设动作过程中没有额外的即时扣分,折扣因子 $\gamma = 1$)

现在我们要计算状态 $s_{\text{桥中}}$ 的价值 $V^\pi(s)$ 。根据最完整的贝尔曼期望方程(Bellman Expectation Equation),我们需要进行两层加权求和(先算客观环境与即时回报,再算主观策略):

$$V^\pi(s) = \sum_{a \in \mathcal{A}} \pi(a | s) \underbrace{\sum_{s' \in \mathcal{S}} p(s' | s, a) [r(s, a) + \gamma V^\pi(s')]}_{Q^\pi(s, a)}$$

由于在这个例子中,即时回报 $r(s, a)$ 仅由当前的动作决定,与下一个状态 s' 无关(因为只要做

出动作就立刻扣分)，我们可以把公式化简并提取出来，形成大家最常用的剥洋葱结构：

$$V^\pi(s) = \sum_{a \in \mathcal{A}} \pi(a | s) \left(r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s' | s, a) V^\pi(s') \right)$$

我们从内到外，像剥洋葱一样来计算：第一步：计算“如果选了不同动作，各自的客观期望值（ Q 值）是多少”在强化学习里，动作价值函数 $Q(s, a)$ 不仅包含未来的长远胜率，还必须包含做这个动作需要当场付出的代价 r 。如果选 $a_{\text{爬}}$ 的期望价值 $Q(s_{\text{桥中}}, a_{\text{爬}})$ ：

$$Q(s_{\text{桥中}}, a_{\text{爬}}) = \underbrace{-10}_{\text{即时体力代价 } r} + 1 \times \left(\underbrace{0.4 \times 100}_{\text{成功概率} \times \text{价值}} + \underbrace{0.6 \times (-100)}_{\text{坠落概率} \times \text{价值}} \right)$$

$$Q(s_{\text{桥中}}, a_{\text{爬}}) = -10 + (40 - 60) = -10 - 20 = -30$$

如果选 $a_{\text{跑}}$ 的期望价值 $Q(s_{\text{桥中}}, a_{\text{跑}})$ ：

$$Q(s_{\text{桥中}}, a_{\text{跑}}) = \underbrace{-2}_{\text{即时时间代价 } r} + 1 \times \left(\underbrace{0.9 \times 100}_{\text{成功概率} \times \text{价值}} + \underbrace{0.1 \times (-100)}_{\text{坠落概率} \times \text{价值}} \right)$$

$$Q(s_{\text{桥中}}, a_{\text{跑}}) = -2 + (90 - 10) = -2 + 80 = +78$$

中间小结：纯看环境与肉体代价，【跑】因为省时且客观胜率高，动作价值高达 +78；【爬】因为过程痛苦且死亡率高，动作价值跌到了 -30。第二步：结合你的主观策略，计算状态的最终价值 $V(s_{\text{桥中}})$ 现在，把你主观选择这两个动作的概率（30% 爬，70% 跑）加权乘上去：

$$V^\pi(s_{\text{桥中}}) = \pi(a_{\text{爬}} | s_{\text{桥中}}) \times Q(s_{\text{桥中}}, a_{\text{爬}}) + \pi(a_{\text{跑}} | s_{\text{桥中}}) \times Q(s_{\text{桥中}}, a_{\text{跑}})$$

$$V^\pi(s_{\text{桥中}}) = (0.3 \times (-30)) + (0.7 \times 78)$$

$$V^\pi(s_{\text{桥中}}) = -9 + 54.6 = +45.6$$

3. 这个计算结果的物理含义最终算出来的状态价值是 +45.6 分。这个数字是一个毫无破绽、完全体的强化学习状态价值。我们可以清晰地看到驱动智能体命运的三大核心力量是如何共同作用的：当下的苟且（即时回报 r ）：你在动作发生瞬间肉体付出的代价（爬的 -10 与跑的 -2）。远方的客观概率（环境转移概率 p ）：大自然物理规律决定的生死概率（成功率 40% 对 90%）。无论你主观怎么想，大自然的结算铁面无私。主观的决策偏好（策略 π ）：在面对“当下的代价”与“远方的风险”时，你个人表现出来的性格偏好（30% 与 70%）。如果你是一个极度胆小的人：把策略改成 100% 选择【爬】（ $\pi(a_{\text{爬}}) = 1.0$ ），那么你在桥中央的现状价值就会瞬间跌落到 $V(s_{\text{桥中}}) = -30$ （不仅死亡率高，一路上爬得还痛苦）。如果你通过学习进化了策略：变得果断，改为 100% 选择【跑】（ $\pi(a_{\text{跑}}) = 1.0$ ），那么你在桥中央的现状价值就会暴涨到 $V(s_{\text{桥中}}) = +78$ 。

通过这个加入 r 后的标准独木桥例子，我们成功拆解了贝尔曼期望方程中所有字母的现实含义。看清了这种“两层求和、代价前置”的结构，再回头看课本上的公式，它就再也不是一堆冰冷的符号，而是一个关于“选择、代价与命运”的数学模型了。

3、动态规划

动态规划 (dynamic programming) 是程序设计算法中非常重要的内容, 能够高效解决一些经典问题, 例如背包问题和最短路径规划。**动态规划的基本思想是将待求解问题分解成若干个子问题, 先求解子问题, 然后从这些子问题的解得到目标问题的解。**动态规划会保存已解决的子问题的答案, 在求解目标问题的过程中, 需要这些子问题答案时就可以直接利用, 避免重复计算。本章介绍如何用动态规划的思想来求解在马尔可夫决策过程中的最优策略。

基于动态规划的强化学习算法主要有两种: 一是策略迭代 (policy iteration), 二是价值迭代 (value iteration)。其中, 策略迭代由两部分组成: 策略评估 (policy evaluation) 和策略提升 (policy improvement)。具体来说, **策略迭代中的策略评估使用贝尔曼期望方程来得到一个策略的状态价值函数, 这是一个动态规划的过程; 而价值迭代直接使用贝尔曼最优方程来进行动态规划, 得到最终的最优状态价值。**动态规划算法的局限在于马尔可夫决策过程已知且有限。对于“已知”: 现实中的白盒环境很少, 我们无法将其运用到很多实际场景中。对于“有限”, 即状态空间和动作空间是离散且有限的, 同样限制了动态规划的运用场景。

3.1 策略迭代算法

策略迭代是策略评估和策略提升不断循环交替, 直至最后得到最优策略的过程。本节分别对这两个过程进行详细介绍。

3.1.1 策略评估

根据 1. 贝尔曼方程的后向归纳原理的总结, 当知道奖励函数和状态转移函数时, 我们可以根据下一个状态的价值来计算当前状态的价值。因此, 根据动态规划的思想, 可以把**计算下一个可能状态的价值当成一个子问题, 把计算当前状态的价值看作当前问题。**在得知子问题的解后, 就可以求解当前问题。(先求解子问题, 再利用子问题求解当前问题)

3.1.2 从时间尺度到算法迭代轮数

从时间推进的因果逻辑来看, 贝尔曼方程给我们的启示应该是“用未来的状态价值来算当前的状态价值。为了写算法, 我们需要将时间轴 (时刻 t) 与计算机算法里的时间轴 (迭代轮数 k) 强行剥离开。

如果游戏里有“环” (比如走桥会滑落, 水流会把人冲回上游), 或者状态多到数不清, 我们根本没有一个现成的、完美的未来价值 $V(s')$ 让我们去直接调用。

第 0 轮 ($k = 0$): 计算机两眼一抹黑, 它随机瞎猜一个价值表格, 比如认为所有格子的价值都是 0。我们把这个瞎猜的表格记为 V^0 。

第 1 轮 ($k = 1$): 计算机想要提升猜测的精度, 计算出一张稍微准一点的新表格, 记为 V^1 。它怎么算呢? 它把旧表格 V^0 拍在桌上, 代入公式的右边:

$$V^1(s) = \sum \pi \left(R + \gamma \sum P \cdot V^0(s') \right)$$

第 2 轮 ($k = 2$): 计算机想要得到更准的表格 V^2 。它把刚才算出来的 V^1 当作基准, 再次

代入公式右边：

$$V^2(s) = \sum \pi \left(R + \gamma \sum P \cdot \mathbf{V}^1(s') \right)$$

以此类推，到了第 $k+1$ 轮，想要更新当前这轮的表格 $V^{k+1}(s)$ ，计算机手里唯一的已知线索，就是上一轮（第 k 轮）已经存好的旧表格 $V^k(s')$ 。所以公式写成了：

$$V^{k+1}(s) = \sum_{a \in A} \pi(a|s) \left(r(s, a) + \gamma \sum_{s' \in S} P(s'|s, a) \mathbf{V}^k(s') \right)$$

因此，“用上一轮（迭代轮数 k ）的状态价值函数，来计算当前这一轮（迭代轮数 $k+1$ ）** 的状态价值函数。“和用未来的状态 s' 算当下的状态 s ”是一致的。另外。可以证明，随着迭代轮数 k 的增长，价值函数序列 $\{V^k\}$ 会收敛到 $\mathbf{V}^\pi$ 。

3.1.3 策略提升

如果在每一个状态 s 下，智能体按照一个新策略 π' 选出的动作，其期望价值（ Q 值）都大于或等于老策略 π 的状态价值（ V 值），那么这个新策略的全局状态价值，就一定大于或等于老策略。

3.2 策略迭代与策略提升在独木桥问题的应用

3.2.1 问题背景

一个智能体（玩家）正面临后有追兵、前有险阻的单向逃生场景。该场景可以用离散马尔可夫决策过程（MDP）进行建模。

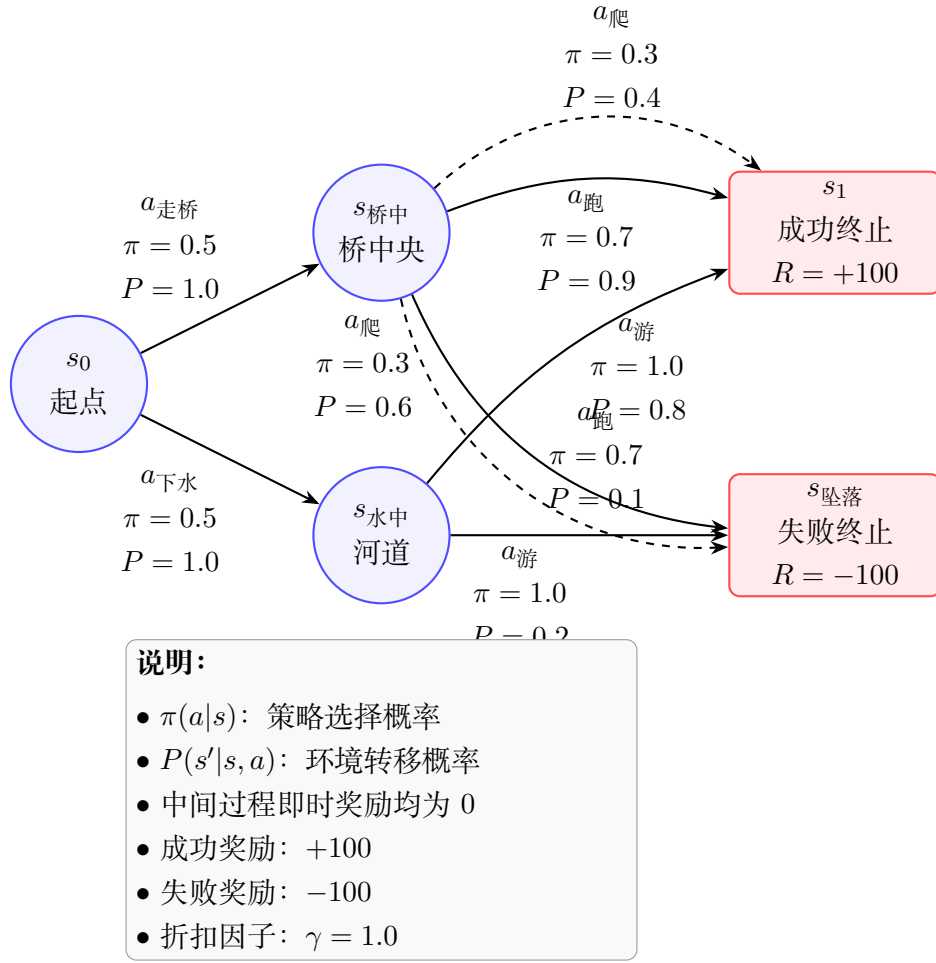


图 1: 独木桥问题的 MDP 状态转移图

3.2.2 MDP 五元组设定

(1) 状态空间 $\mathcal{S}$ 系统状态定义如下:

$$\mathcal{S} = \{s_0, s_{\text{桥中}}, s_{\text{水中}}, s_{\text{坠落}}, s_1\}$$

其中:

- s_0 : 起点 (桥头)
- $s_{\text{桥中}}$: 桥中央状态
- $s_{\text{水中}}$: 河道状态
- $s_{\text{坠落}}$: 失败终止状态
- s_1 : 成功逃脱终止状态

(2) **动作空间 $\mathcal{A}$** 不同状态下的动作集合定义为：

$$\mathcal{A}(s_0) = \{a_{\text{走桥}}, a_{\text{下水}}\}$$

$$\mathcal{A}(s_{\text{桥中}}) = \{a_{\text{爬}}, a_{\text{跑}}\}$$

$$\mathcal{A}(s_{\text{水中}}) = \{a_{\text{游}}\}$$

(3) **状态转移概率** 起点状态的确定性转移：

$$P(s_{\text{桥中}}|s_0, a_{\text{走桥}}) = 1.0$$

$$P(s_{\text{水中}}|s_0, a_{\text{下水}}) = 1.0$$

桥中央状态的随机转移规律：

- 若选择动作 $a_{\text{爬}}$

$$P(s_{\text{坠落}}|s_{\text{桥中}}, a_{\text{爬}}) = 0.6$$

$$P(s_1|s_{\text{桥中}}, a_{\text{爬}}) = 0.4$$

- 若选择动作 $a_{\text{跑}}$

$$P(s_1|s_{\text{桥中}}, a_{\text{跑}}) = 0.9$$

$$P(s_{\text{坠落}}|s_{\text{桥中}}, a_{\text{跑}}) = 0.1$$

河道中的随机转移规律：

$$P(s_1|s_{\text{水中}}, a_{\text{游}}) = 0.8$$

$$P(s_{\text{坠落}}|s_{\text{水中}}, a_{\text{游}}) = 0.2$$

(4) **奖励函数** 所有中间过程即时奖励均为：

$$R(s, a, s') = 0$$

终止状态奖励如下：

$$R(s, a, s_1) = +100$$

$$R(s, a, s_{\text{坠落}}) = -100$$

(5) 折扣因子

$$\gamma = 1.0$$

即未来奖励不进行折扣。

3.2.3 当前待评估策略 π

当前策略定义如下：

在起点状态：

$$\pi(a_{\text{走桥}}|s_0) = 0.5$$

$$\pi(a_{\text{下水}}|s_0) = 0.5$$

在桥中央状态：

$$\pi(a_{\text{爬}}|s_{\text{桥中}}) = 0.3$$

$$\pi(a_{\text{跑}}|s_{\text{桥中}}) = 0.7$$

在河道状态：

$$\pi(a_{\text{游}}|s_{\text{水中}}) = 1.0$$

3.2.4 策略评估理论

根据贝尔曼期望方程，策略评估的迭代公式为：

$$V^{k+1}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} P(s'|s, a) [R(s, a, s') + \gamma V^k(s')] \quad (1)$$

由于本题中：

$$R(s, a, s') = 0$$

且：

$$\gamma = 1$$

终止状态价值固定为：

$$V(s_1) = 100$$

$$V(s_{\text{坠落}}) = -100$$

因此可简化为：

$$V^{k+1}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} P(s'|s, a) V^k(s') \quad (2)$$

3.2.5 第 1 轮策略评估

初始化：

$$V^0(s_0) = 0$$

$$V^0(s_{\text{桥中}}) = 0$$

$$V^0(s_{\text{水中}}) = 0$$

(1) 计算桥中央状态价值

$$\begin{aligned} V^1(s_{\text{桥中}}) &= 0.3 [0.4 \times 100 + 0.6 \times (-100)] \\ &\quad + 0.7 [0.9 \times 100 + 0.1 \times (-100)] \\ &= 0.3 \times (-20) + 0.7 \times 80 \\ &= -6 + 56 \\ &= 50 \end{aligned} \quad (3)$$

(2) 计算河道状态价值

$$\begin{aligned} V^1(s_{\text{水中}}) &= 1.0 [0.8 \times 100 + 0.2 \times (-100)] \\ &= 80 - 20 \\ &= 60 \end{aligned} \quad (4)$$

(3) 计算起点状态价值 由于采用同步雅可比迭代，因此只能调用旧值：

$$\begin{aligned}
V^1(s_0) &= 0.5 \times V^0(s_{\text{桥中}}) + 0.5 \times V^0(s_{\text{水中}}) \\
&= 0.5 \times 0 + 0.5 \times 0 \\
&= 0
\end{aligned} \tag{5}$$

因此：

$$V^1 = [0, 50, 60]$$

即：

$$[V(s_0), V(s_{\text{桥中}}), V(s_{\text{水中}})] = [0, 50, 60]$$

3.2.6 第 2 轮策略评估

(1) 桥中央状态

$$\begin{aligned}
V^2(s_{\text{桥中}}) &= 0.3 \times (-20) + 0.7 \times 80 \\
&= 50
\end{aligned} \tag{6}$$

(2) 河道状态

$$V^2(s_{\text{水中}}) = 60 \tag{7}$$

(3) 起点状态

$$\begin{aligned}
V^2(s_0) &= 0.5 \times V^1(s_{\text{桥中}}) + 0.5 \times V^1(s_{\text{水中}}) \\
&= 0.5 \times 50 + 0.5 \times 60 \\
&= 25 + 30 \\
&= 55
\end{aligned} \tag{8}$$

因此：

$$V^2 = [55, 50, 60]$$

即：

$$[V(s_0), V(s_{\text{桥中}}), V(s_{\text{水中}})] = [55, 50, 60]$$

3.2.7 收敛结果分析

继续策略迭代后：

$$V^{k+1}(s) = V^k(s)$$

状态价值不再变化，因此算法收敛。

最终状态价值函数为：

$$V^\pi(s_0) = 55$$

$$V^\pi(s_{\text{桥中}}) = 50$$

$$V^\pi(s_{\text{水中}}) = 60$$

3.2.8 基于策略迭代的策略改进

在完成策略评估后，得到当前策略对应的状态价值函数：

$$V^\pi(s_0) = 55$$

$$V^\pi(s_{\text{桥中}}) = 50$$

$$V^\pi(s_{\text{水中}}) = 60$$

随后进入策略改进（Policy Improvement）阶段。

策略改进的核心思想为：

$$\pi'(s) = \arg \max_a Q^\pi(s, a) \quad (9)$$

其中：

$$Q^\pi(s, a) = \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^\pi(s')] \quad (10)$$

由于本题中：

$$\gamma = 1$$

且下一步直接到达终止状态，因此动作价值可直接计算。

(1) 桥中央状态的策略改进 对于动作 $a_{\text{爬}}$ ：

$$\begin{aligned}
Q^\pi(s_{\text{桥中}}, a_{\text{爬}}) &= 0.4 \times 100 + 0.6 \times (-100) \\
&= 40 - 60 \\
&= -20
\end{aligned} \tag{11}$$

对于动作 $a_{\text{跑}}$ ：

$$\begin{aligned}
Q^\pi(s_{\text{桥中}}, a_{\text{跑}}) &= 0.9 \times 100 + 0.1 \times (-100) \\
&= 90 - 10 \\
&= 80
\end{aligned} \tag{12}$$

因此：

$$Q^\pi(s_{\text{桥中}}, a_{\text{跑}}) > Q^\pi(s_{\text{桥中}}, a_{\text{爬}})$$

故改进后的策略应选择：

$$\pi'(a_{\text{跑}}|s_{\text{桥中}}) = 1.0$$

$$\pi'(a_{\text{爬}}|s_{\text{桥中}}) = 0$$

此时桥中央状态的改进后价值变为：

$$V^{\pi'}(s_{\text{桥中}}) = 80$$

(2) 起点状态的策略改进 接下来比较起点状态的两个动作价值。

若选择动作 $a_{\text{走桥}}$ ：

$$\begin{aligned}
Q^\pi(s_0, a_{\text{走桥}}) &= V^{\pi'}(s_{\text{桥中}}) \\
&= 80
\end{aligned} \tag{13}$$

若选择动作 $a_{\text{下水}}$ ：

$$\begin{aligned}
Q^\pi(s_0, a_{\text{下水}}) &= V^\pi(s_{\text{水中}}) \\
&= 60
\end{aligned} \tag{14}$$

因此：

$$Q^{\pi}(s_0, a_{\text{走桥}}) > Q^{\pi}(s_0, a_{\text{下水}})$$

说明桥路在策略改进后优于水路。
故起点状态的最优策略应调整为：

$$\pi'(a_{\text{走桥}}|s_0) = 1.0$$

$$\pi'(a_{\text{下水}}|s_0) = 0$$

(3) 最终改进策略 最终改进后的确定性策略为：

$$\pi^*(a_{\text{走桥}}|s_0) = 1.0$$

$$\pi^*(a_{\text{跑}}|s_{\text{桥中}}) = 1.0$$

$$\pi^*(a_{\text{游}}|s_{\text{水中}}) = 1.0$$

此时系统最优状态价值为：

$$V^{\pi^*}(s_0) = 80$$

相比初始随机策略：

$$55 \rightarrow 80$$

说明策略改进后智能体的生存收益显著提升。

3.3 价值迭代算法

策略迭代中的策略评估需要进行很多轮才能收敛得到某一策略的状态函数，这需要很大的计算量，尤其是在状态和动作空间比较大的情况下。我们是否必须要完全等到策略评估完成后再进行策略提升呢？试想一下，可能出现这样的情况：虽然状态价值函数还没有收敛，但是不论接下来怎么更新状态价值，策略提升得到的都是同一个策略。如果只在策略评估中进行一轮价值更新，然后直接根据更新后的价值进行策略提升，这样是否可以呢？答案是肯定的，这其实就是本节将要讲解的**价值迭代算法**，它可以被认为是一种策略评估只进行了一轮更新的策略迭代算法。需要注意的是，价值迭代中不存在显式的策略，我们只维护一个状态价值函数。

简单地讲，策略迭代 (Policy Iteration)：两步走。每轮都必须把表格算到彻底收敛（完美不准变成完美准），才敢做一次策略提升。价值迭代 (Value Iteration)：一步走。急脾气，表格只刷一轮（不管准不准），就地立刻一边提升策略，一边更新价值。

策略迭代（策略评估部分）的公式：

$$V^{k+1}(s) = \sum_{a \in \mathcal{A}} \pi(a | s) \left(R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' | s, a) V^k(s') \right)$$

特点：公式里有老老实实的 $\sum_a \pi(a | s)$ 。你策略规定是 50/50 扔硬币，它就老老实实帮你按 50/50 的加权去算期望。所以它要算很多轮才能把这个糊涂蛋策略的真实价值算清楚。价值迭代的公式：

$$V^{k+1}(s) = \max_{a \in \mathcal{A}} \left(R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' | s, a) V^k(s') \right)$$

特点：公式里的 $\sum_a \pi(a | s)$ 消失了！取而代之的是一个极其霸道的 $\max_a$ 。物理直觉：计算机在刷新状态 s 的价值时，它会把【爬】、【跑】、【下水】等所有可选动作的未来期望（即 Q 值）全部算一遍，然后只挑那个最大的数（ $\max$ ）作为 s 的新价值。它默认你下一次一定会选最好的动作。

参考 3.2 节用“独木桥”题目看两者的计算差异。我们回到刚才全 0 的初始表格 V^0 。看看价值迭代第一轮会怎么算起点 s_0 的价值：如果是策略迭代（策略评估）：第一轮里，因为公式右边查旧表 $V^0(s) = 0, V^0(s) = 0$ 。配合 50/50 的策略，算出来的 $V^1(s_0) = 0.5 \times 0 + 0.5 \times 0 = 0$ 。终端的奖励信号（Reward Signal）尚未反向传播至起点。如果是价值迭代：在刷新表格 V^{k+1} 时，它会同时算所有状态。在计算 s_0 时，它直接用 $\max$ 斩断了老策略的纠结：

$$V^1(s_0) = \max \left\{ \text{走桥的未来, 走水路的未来} \right\}$$

虽然第一轮它算出来的结果可能也是 0（因为终端的奖励信号（Reward Signal）尚未反向传播至起点），但关键在于它的进化速度。当信息从终点往回传导时：策略迭代必须等中间状态的价值完全“稳定”了，起点才去挑好路。价值迭代则是每个格子一边在接收终点传过来的信息，一边在脑子里自动做 $\max$ 筛选。当价值表格经过一轮轮 $\max$ 刷新彻底不再变化（收敛）时，最优策略和最优价值同时被一次性解了出来。

3.3.1 基于价值迭代的最优解法

价值迭代（Value Iteration）不再固定策略，而是在每一轮迭代中直接选择当前状态下的最优动作。

其贝尔曼最优方程为：

$$V^{k+1}(s) = \max_a \sum_{s'} P(s' | s, a) \left[R(s, a, s') + \gamma V^k(s') \right] \quad (15)$$

由于：

$$\gamma = 1$$

且终止状态价值固定：

$$V(s_1) = 100$$

$$V(s_{\text{坠落}}) = -100$$

因此可直接进行迭代。

(1) 初始化

$$V^0(s_0) = 0$$

$$V^0(s_{\text{桥中}}) = 0$$

$$V^0(s_{\text{水中}}) = 0$$

(2) 第 1 轮价值迭代 桥中央状态：

$$\begin{aligned} V^1(s_{\text{桥中}}) &= \max\{-20, 80\} \\ &= 80 \end{aligned} \tag{16}$$

河道状态：

$$V^1(s_{\text{水中}}) = 60 \tag{17}$$

起点状态由于同步更新，只能读取旧值：

$$\begin{aligned} V^1(s_0) &= \max\{V^0(s_{\text{桥中}}), V^0(s_{\text{水中}})\} \\ &= 0 \end{aligned} \tag{18}$$

因此：

$$V^1 = [0, 80, 60]$$

(3) 第 2 轮价值迭代 桥中央状态保持不变：

$$V^2(s_{\text{桥中}}) = 80$$

河道状态保持不变：

$$V^2(s_{\text{水中}}) = 60$$

起点状态：

$$\begin{aligned} V^2(s_0) &= \max\{80, 60\} \\ &= 80 \end{aligned} \tag{19}$$

因此：

$$V^2 = [80, 80, 60]$$

算法已收敛。

(4) 价值迭代得到的最优策略 根据最优动作选择规则：

$$\pi^*(s) = \arg \max_a Q(s, a)$$

可得：

在桥中央状态：

$$\pi^*(a_{\text{跑}}|s_{\text{桥中}}) = 1.0$$

在起点状态：

$$\pi^*(a_{\text{走桥}}|s_0) = 1.0$$

因此最终最优策略为：

$$\pi^*(a_{\text{走桥}}|s_0) = 1.0$$

$$\pi^*(a_{\text{跑}}|s_{\text{桥中}}) = 1.0$$

对应最优状态价值：

$$V^*(s_0) = 80$$

(5) 策略迭代与价值迭代的区别 策略迭代的过程为：

策略评估 → 策略改进

即：

- 先固定策略计算 V^π
- 再基于 Q^π 改进策略

而价值迭代则直接在每轮迭代中执行：

$$\max_a$$

因此其本质为：

策略评估 + 策略改进

的同步融合。

本案例中：

- 策略迭代先得到：

$$V^\pi(s_0) = 55$$

- 再经过策略改进得到：

$$V^{\pi^*}(s_0) = 80$$

- 价值迭代则直接收敛到：

$$V^*(s_0) = 80$$

4、强化学习与现代控制理论的底层数学对偶性

在学习研究对离散 MDP 进行策略评估时，我注意到一个核心现象：在迭代初期，起点（初始状态）的价值估计保持为 0，直至迭代轮数 k 达到或超过起点到终端状态的拓扑路径长度时，起点的价值才被“照亮”。这一现象在计算机科学中被称为“价值信号的反向传播（Backward Propagation）”引起的延迟；但若将其置于现代控制理论的视界下审视，其本质与连续时间动态系统中的终端边界条件解算完全对偶。这两个原本平行发展的学科，在底层数学机制上表现出了惊人的巧合：

1. 终端边界条件的本质一致在强化学习（RL）中：我们将环境边界死死锚定在特定的终止状态（如成功逃脱或失稳坠落），通过明码标价的终端奖励/惩罚（Terminal Reward/Cost）作为整张价值网络演化的引力源。在现代控制理论中：这完全等价于控制系统给定的终端状态约束（Terminal Constraint）。例如在航天器姿态控制或导弹制导问题中，我们严格限定在终端时刻 t_f 时，姿态跟踪误差为 0、角速度为 0：

$$x(t_f) = \mathbf{0}$$

2. 时间反向解算与信息流动的巧合强化学习通过贝尔曼方程进行 $k \rightarrow k+1$ 轮的同步迭代更新，在控制系统中有着完美的连续时间数学对应，即哈密顿-雅可比-贝尔曼方程（HJB Equation）或最优控制中的后向动态规划（Backward Dynamic Programming）。在求解有限时域的线性二次型调节器（LQR）或最优控制问题时，状态值函数或控制增益的求解依赖于伴随状态（微分黎卡提方程 Riccati Equation）的后向积分（Backward Integration）：

$$-\dot{P}(t) = A^T P(t) + P(t)A - P(t)BR^{-1}B^T P(t) + Q, \quad \text{边界条件: } P(t_f) = P_f$$

这种解算过程形成了跨学科的惊人巧合：

控制系统的未解出状态：由于黎卡提方程或 HJB 方程必须从终端时间 t_f 开始沿着时间轴向后倒着积分，当倒推的积分时间长度 $\Delta t = t_f - t$ 还没有累积覆盖到初始时刻 t_0 时，初始时刻（起点）的控制律增益和状态价值函数在数学上就必然处于“未解出（未收敛）”的状态。

强化学习的信息延迟：这恰好完美解释了为何在离散策略评估中，终端的奖励信号需要经过一轮轮迭代才能反向传播到起点。RL 表格中“起点的光还没亮”，在物理控制本质上就是“后向积分时间尚未推进至初始时刻 t_0 ”的离散时序投影。

5、蒙特卡洛算法

蒙特卡洛方法（Monte-Carlo methods）也被称为统计模拟方法，是一种基于概率统计的数值计算方法。运用蒙特卡洛方法时，我们通常使用重复随机抽样，然后运用概率统计方法来从抽样结果中归纳出我们想求的目标的数值估计。一个简单的例子是用蒙特卡洛方法来计算圆的面积。例如，在图 3-5 所示的正方形内部随机产生若干个点的个数，细数落在圆中点的个数，圆的面积与正方形面积之比就等于圆中点的个数与正方形中点的个数之比。如果我们随机产生的点的个数越多，计算得到圆的面积就越接近于真实的圆的面积。

用蒙特卡洛方法来估计一个策略在一个马尔可夫决策过程中的状态价值函数。回忆一下，一个状态的价值是它的期望回报，那么一个很直观的想法就是用策略在 MDP 上采样很多条序列，计算从这个状态出发的回报再求其期望就可以了，公式如下：

$$V^\pi(s) = \mathbb{E}_\pi[G_t | S_t = s] \approx \frac{1}{N} \sum_{i=1}^N G_t^{(i)}$$

在一条序列中，可能没有出现过这个状态，可能只出现过一次这个状态，也可能出现过很多次这个状态。我们介绍的蒙特卡洛价值估计方法会在该状态每一次出现时计算它的回报。还有一种选择是一条序列只计算一次回报，也就是这条序列第一次出现该状态时计算后面的累积奖励，而后面再次出现该状态时，该状态就被忽略了。

5.1 序列采样的定义

在强化学习中，采样 N 遍，是指让策略 π 在环境里自由地、盲目地从头到尾玩 N 局。以上面那个“独木桥”问题为例，假设在起点 s_0 ，策略 π 规定：走桥的概率 $\pi(a_{\text{走桥}} | s_0) = 0.8$ 下水的概率 $\pi(a_{\text{下水}} | s_0) = 0.2$ 所谓的采样一次，是指智能体站在起点 s_0 ，写代码时我们调用一个随机数生成器（比如 Python 里的 `numpy.random.choice`），根据 $[0.8, 0.2]$ 的概率强制作出一个决定。它有 80% 的可能摇号摇到“走桥”；有 20% 的可能摇号摇到“下水”。智能体一旦做出动作，就会一路往下走，直到游戏结束（终止状态）。这一整条从头到尾的轨迹，叫做“采样了一条序列（Episode）”。

5.2 蒙特卡洛算法流程

假设我们现在用策略 π 从状态 s 开始采样序列，据此来计算状态价值。我们为每一个状态维护一个计数器和总回报，计算状态价值的具体过程如下所示。

(1) 使用策略 π 采样若干条序列：

$$s_0^{(i)} \xrightarrow{a_0^{(i)}} r_0^{(i)}, s_1^{(i)} \xrightarrow{a_1^{(i)}} r_1^{(i)}, s_2^{(i)} \rightarrow \dots \xrightarrow{a_{T-1}^{(i)}} r_{T-1}^{(i)}, s_T^{(i)}$$

(2) 对每一条序列中的每一时间步 t 的状态 s 进行以下操作：更新状态 s 的计数器 $N(s) \leftarrow N(s) + 1$ ；更新状态 s 的总回报 $M(s) \leftarrow M(s) + G_t$ ；

(3) 每一个状态的价值被估计为回报的平均值 $V(s) = M(s)/N(s)$ 。根据大数定律，当 $N(s) \rightarrow \infty$ ，有 $V(s) \rightarrow V^\pi(s)$ 。计算回报的期望时，除了可以把所有的回报加起来除以次数。

(4) 但对于上述方法，如果游戏要跑 1000 万局，把每一局的 G 都加到 $M(s)$ 里，数字会变得巨大无比，容易导致内存溢出或者数值越界。等价的，引出增量更新公式：它不需要知道过去的所有总和，它只用当前的旧价值 $V(s)$ 和这一局刚刚拿到的新回报 G 做对比。对于每个状态 s 和对应回报 G ，进行如下计算：

$$N(s) \leftarrow N(s) + 1$$

$$V(s) \leftarrow V(s) + \frac{1}{N(s)}(G - V(s))$$

即

$$\text{新价值} = \text{旧价值} + \frac{1}{\text{看过的次数}} \times (\text{这一次的实际得分} - \text{过去的平均预期})$$

如果 $(G - V(s)) > 0$ （说明这一局的表现比以往的平均期望要好），那么价值 $V(s)$ 就往上加一点。而且随着你看过的次数 $N(s)$ 越来越多， $\frac{1}{N(s)}$ 就会越来越小。这意味着：当你已经是见过大风大浪的老司机时（ N 很大），单单一两局的成功或失败（ G ），已经很难大幅度撼动你对这个状态的认知了（更新幅度变小）。

6、时序差分算法

6.1 无模型强化学习

动态规划算法要求马尔可夫决策过程是已知的，即要求与智能体交互的环境是完全已知的（例如迷宫或者给定规则的网格世界）。在此条件下，智能体其实并不需要和环境真正交互来采样数据，直接用动态规划算法就可以解出最优价值或策略。但对于大部分强化学习现实场景（例如电子游戏或者一些复杂物理环境），其马尔可夫决策过程的状态转移概率是无法写出来的，也就无法直接进行动态规划。在这种情况下，智能体只能和环境进行交互，通过采样到的数据来学习，这类学习方法统称为无模型的强化学习（model-free reinforcement learning）。无模型的强化学习算法不需要事先知道环境的奖励函数和状态转移函数，而是直接使用和环境交互的过程中采样到的数据来学习，这使得它可以被应用到一些简单的实际场景中。无模型的强化学习中的两大经典算法：Sarsa 和 Q-learning，它们都是基于时序差分（temporal difference, TD）的强化学习算法。

在上面独木桥的例子中，无论是即时回报 r （爬扣 10 分，跑扣 2 分），还是环境转移概率 p （爬有 60% 概率坠落），都是我们（算法设计者）提前规定好、写死在代码里的。在强化学习的术语里，这就叫做“环境模型（Model）”已经完全知晓。 $r(s, a)$ 是我们规定的（奖励函数设计）：大自然怎么打分，是我们作为上帝视角定义的规则。 $p(s' | s, a)$ 是已经建好模的（转移概

率): 风速多大、摩擦力多少, 我们已经用物理公式或概率公式算得清清楚楚。Q-learning (以及整个 Model-free 强化学习) 就是为了解决这种“无法建模”的问题而生的。

6.2 时序差分算法原理

简单来说: TD 是 MC 的“精神续作”, 但它用了一种极其聪明的数学手段, 把 MC 的致命缺点给干掉了。在上一节的蒙特卡洛 (MC) 中, 我们说它是“秋后算账”: 只要你看过了次数 $N(s)$ 足够多, 你就用新序列的总得分 G 去修正旧价值:

$$V(s_t) \leftarrow V(s_t) + \alpha[G_t - V(s_t)]$$

这里的核心问题就在于这个 G_t (总回报)。为了拿到 G_t , 智能体必须一路走到黑 (要么安全过桥 s_1 , 要么掉下悬崖 $s_{\text{坠落}}$)。如果这个独木桥不是一步就能走完, 而是有 1000 个台阶的长桥, 智能体就必须老老实实走完 1000 步拿到最后的输赢, 才能回过头来更新“第 1 步”的价值。如果智能体在第 999 步卡住了, 或者这个任务是个无限循环 (比如自动驾驶汽车要在环形跑道上一直开), 它永远拿不到最终的 G , 那么 MC 算法就会彻底瘫痪。

时序差分 (TD) 算法审时度势, 它把 MC 更新公式里的那个必须走到终点才能算出来的 G_t , 换成了一个“眼前的即时回报 + 下一步的预估价值”:

$$V(s_t) \leftarrow V(s_t) + \alpha(r_{t+1} + \gamma V(s_{t+1}) - V(s_t))$$

被替换掉的这一项: $r_{t+1} + \gamma V(s_{t+1})$ 叫做 TD 目标值 (TD Target)。它代表: 我今天刚拿到的肉体代价 r_{t+1} , 加上我站在新的位置上、对未来长远价值的当下预估 $\gamma V(s_{t+1})$ 。 α 是学习率, 相当于书上写的 $\frac{1}{N(s)}$, 表示我们拨动原有认知的幅度。

既然我们可以用时序差分方法来估计价值函数, 那一个很自然的问题是, 我们能否用类似策略迭代的方法来进行强化学习。策略评估已经可以通过时序差分算法实现, 那么在不知道奖励函数和状态转移函数的情况下该怎么进行策略提升呢? **答案是时可以直接用时序差分算法来估计行动价值函数 (这同时也是第 7 节中提到的 SARSA 算法针对动作价值 Q 而非状态价值 V)**:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

然后我们用贪婪算法来选取在某个状态下动作价值最大的那个动作, 即 $\arg \max_a Q(s, a)$ 。这样似乎已经形成了一个完整的强化学习算法: 用贪婪算法根据动作价值选取动作来和环境交互, 再根据得到的数据用时序差分算法来更新动作价值估计。然而这个简单的算法存在两个需要进一步考虑的问题。第一, 如果要用时序差分算法来准确地估计策略的状态价值函数, 我们需要用极大量的样本来进行更新。但实际上我们可以忽略这一点, 直接用一些样本来评估策略, 然后就可以更新策略了。我们可以这么做的原因是策略提升可以在策略评估未完全进行的情况下进行, 回顾一下, 价值迭代 (参见 4.4 节) 就是这样, 这其实是广义策略迭代 (generalized policy iteration) 的思想。第二, 如果在策略提升中一直根据贪婪算法得到一个确定性策略, 可能会导致某些状态动作对 (s, a) 永远没有在序列中出现, 以至于无法对其动作价值进行估计, 进而无法保证策略提升后的策略比之前的好。我在第一节总结对此有详细讨论。简单常用的解决方案是不再一味使用贪婪算法, 而是采用一个 ϵ -贪婪策略: 有 $1 - \epsilon$ 的概率采用动作价值最大的那个

动作，另外有 ϵ 的概率从动作空间中随机采取一个动作，其公式表示为：

$$\pi(a | s) = \begin{cases} \epsilon/|\mathcal{A}| + 1 - \epsilon & \text{如果 } a = \arg \max_{a'} Q(s, a') \\ \epsilon/|\mathcal{A}| & \text{其他动作} \end{cases}$$

对比来看，价值迭代 (Value Iteration) 与 TD (时序差分) 神似：它们都是“急功近利”的。价值迭代每轮只让状态价值往前看一步（利用贝尔曼最优算子逼近一步），不关心当前策略是不是完整的、好不好，只要看到更好的苗头就立刻修正价值；而 TD 也是每走一步，看到下一个状态的苗头，就立刻当场修正价值。策略迭代 (Policy Iteration) 与 MC (蒙特卡洛) 神似：它们都是“谋定而后动”的。策略迭代在“策略评估”阶段，必须老老实实把当前策略下的 V 值彻底算到收敛（全局算透），然后才去整体提升策略；而 MC 也是必须让策略把一局游戏彻底跑完（看到全局终点），拿到最终的总分，才回过头来修正价值。算法对比图如表 1 所示。

表 1: 强化学习与动态规划全景四象限图

维度	单步前瞻（只看一步预测）	完整序列（看整条轨迹结局）
上帝视角 (Model-based / 满概率 全树展开)	动态规划：价值/策略迭代的一步评估 在脑海中全盘推演下一步所有分叉的概率分布。	穷举路径推演 在脑海中全盘推演所有可能的剧本直到游戏结束。
凡人肉身 (Model-free / 环境单线 采样试错)	时序差分算法 (TD) 肉身环境中只走一步，看到一个风景就当场修正认知。	蒙特卡洛算法 (MC) 肉身环境中一路走到黑，直到秋后算账、看重最终结局。

7、SARSA 算法

SARSA 是时序差分 (TD) 流派在“无模型 (Model-free)”环境下的典型化现。它其实是智能体在环境中更迭价值时，经历的五个因果元素的连拼：

S：当前状态 (Current State)

A：当前采取的动作 (Current Action)

R：大自然给的即时回报 (Reward)

S'：到达的下一个状态 (Next State)

A'：在下一个状态下，主观策略决定要采取的下一个动作 (Next Action)

继续用独木桥为例。此时，环境无法建模（不知道大自然的脚滑概率 p ），智能体手里拿着一张 Q 表格 (Q-table)，里面记录着它对每个“状态-动作对”的模糊印象。初始时，所有的 Q 值都是瞎猜的 0： $Q(s_{\text{桥中}}, a_{\text{跑}}) = 0, Q(s_{\text{安全桥尾}}, a_{\text{欢呼}}) = 0$ （已知边界：到达安全桥尾后，只要做出

“欢呼”动作，游戏通关并获得终点价值 +100) 现在，一个 SARSA 智能体来到了桥中央：【S】：智能体当前处于 $s_{\text{桥中}}$ 。【A】：它根据自己的主观策略（比如用 ϵ -greedy 策略，有 70% 概率选跑），决定采取动作 $a_{\text{跑}}$ 。【R】：它拔腿就跑，大自然立刻结算肉体时间代价 $r = -2$ 。【S'】：它运气好没脚滑，身体瞬移到了下一个状态 $s_{\text{安全桥尾}}$ 。【A'】：【关键来了!】站在 $s_{\text{安全桥尾}}$ 的节点上，智能体还没有真正做动作，但它根据自己当下的主观策略，提前预选了下一步的动作—— $a_{\text{欢呼}}$ 。此时，智能体手里集齐了 (S, A, R, S', A') 五张牌，SARSA 算法立刻叫停游戏，当场原地更新 $Q(s_{\text{桥中}}, a_{\text{跑}})$ 的价值：

$$Q(S, A) \leftarrow Q(S, A) + \alpha \left[\underbrace{R + \gamma Q(S', A')}_{\text{TD 目标值}} - Q(S, A) \right]$$

我们带入独木桥的数据（假设学习率 $\alpha = 0.5$, $\gamma = 1$ ）：

$$Q(s_{\text{桥中}}, a_{\text{跑}}) \leftarrow 0 + 0.5 \times [(-2) + 1 \times Q(s_{\text{安全桥尾}}, a_{\text{欢呼}}) - 0]$$

$$Q(s_{\text{桥中}}, a_{\text{跑}}) \leftarrow 0 + 0.5 \times [-2 + 100 - 0] = +49$$

更新完成后，游戏继续。在下一步，智能体将严格执行它刚刚预选好的动作 $a_{\text{欢呼}}$ （即把当前的 S', A' 变成下一轮的 S, A ），循环往复。

8、离线策略算法 Q-Learning

称采样数据的策略为行为策略（behavior policy），称用这些数据来更新的策略为目标策略（target policy）。在线策略（on-policy）算法表示行为策略和目标策略是同一个策略；而离线策略（off-policy）算法表示行为策略和目标策略不是同一个策略。Sarsa 是典型的在线策略算法，而 Q-learning 是典型的离线策略算法。

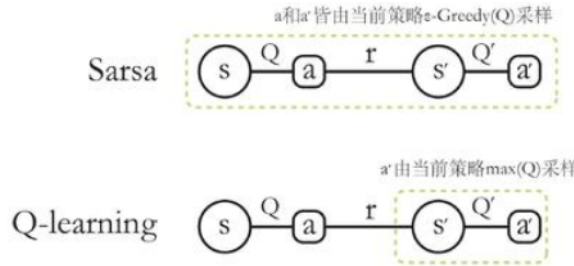


图 2: Sarsa 和 Q-learning 的对比

Q-learning 的更新公式。它不需要 SARSA 的第五张牌（下一步实际要做的动作 A' ），它只看下一步状态 S' 的最大潜力：

$$Q(S, A) \leftarrow Q(S, A) + \alpha \left[\underbrace{R + \gamma \max_{a'} Q(S', a')}_{\text{TD 目标值}} - Q(S, A) \right]$$

你仔细看这关键的半句：

$$\max_{a'} Q(S', a')$$

它的意思是：无论我等会儿在下一个位置 S' 实际上会因为“随机探索”做出多么愚蠢、多么怂的动作，今天我站在当前位置 S 更新价值时，我都假设我等会儿在 S' 绝对能做出最完美、得分最高的动作。

让 Q-learning 智能体穿上鞋子，去未知的独木桥上肉身试错。此时，Q 表格 (Q-table) 里关于桥中央的印象依然是瞎猜的初始值 0： $Q(s_{\text{桥中}}, a_{\text{跑}}) = 0$ ， $Q(s_{\text{安全桥尾}}, a_{\text{欢呼}}) = +100$ ， $Q(s_{\text{安全桥尾}}, a_{\text{发呆}}) = 0$ （假设在桥尾发呆没有分）（设置学习率 $\alpha = 0.5$ ， $\gamma = 1$ ）某一局游戏开始了：**【S】**：智能体站在 $s_{\text{桥中}}$ 。**【A】**：它主观策略爆发，决定采取动作 $a_{\text{跑}}$ 。**【R】**：它拔腿飞奔，肉体当场承受时间代价 $r = -2$ 。**【S'】**：它运气很好，没有脚滑，身体瞬移到了下一个状态 $s_{\text{安全桥尾}}$ 。关键反转：Q-learning 现场更新此时，智能体站在 $s_{\text{安全桥尾}}$ ，它还没有做动作。如果是 SARSA：它必须查一下策略，看看等会儿大步欢呼还是原地发呆。如果策略说有概率发呆，SARSA 就会把发呆的低分也算进去。如果是 Q-learning：它根本不看策略。它直接扫视自己在 $s_{\text{安全桥尾}}$ 的所有备选动作价值：**【欢呼】** 是 +100，**【发呆】** 是 0。它眼里只有最完美的那个：

$$\max_{a'} Q(s_{\text{安全桥尾}}, a') = Q(s_{\text{安全桥尾}}, a_{\text{欢呼}}) = +100$$

现在，Q-learning 掏出公式，当场就地更新：

$$Q(s_{\text{桥中}}, a_{\text{跑}}) \leftarrow Q(s_{\text{桥中}}, a_{\text{跑}}) + \alpha \left[r + \gamma \max_{a'} Q(s_{\text{安全桥尾}}, a') - Q(s_{\text{桥中}}, a_{\text{跑}}) \right]$$

$$Q(s_{\text{桥中}}, a_{\text{跑}}) \leftarrow 0 + 0.5 \times \left[(-2) + 1 \times 100 - 0 \right]$$

$$Q(s_{\text{桥中}}, a_{\text{跑}}) \leftarrow +49$$

8.1 SARSA 算法和 Q-Learning 算法差异

刚才 SARSA 算出来的也是 +49，Q-learning 算出来的也是 +49，那它们有什么本质不同？”因为独木桥在安全桥尾的局势太简单了。如果我们把这个例子稍微改残酷一点，两者的本性就会彻底暴露！

残酷改版：独木桥旁边有“悬崖惊魂带”假设在桥的某一截（状态 $s_{\text{惊魂}}$ ），有两个动作：动作**【走安全内道】**：即时回报 $r = -5$ 。绝对安全，100% 待在桥上。动作**【走悬崖边缘】**：即时回报 $r = +2$ （风景极好，速度极快）。但大自然物理规律决定，走这条路有 10% 的概率失足摔死。现在，我们让两个智能体在训练时，都使用 ϵ -greedy 策略（即 90% 的时间选择当前最优动作，10% 的时间随机乱尝试其他动作用于探索）。

老实人 SARSA 的抉择：当 SARSA 评估“走悬崖边缘”这个动作时，它必须要考虑下一步。它心想：“虽然悬崖边缘很爽，但我等会儿（下一个时间步）有 10% 的概率会瞎摸索，万一我手贱去作死怎么办？”因为 SARSA 把自己未来可能随机乱选、作死的风险实时地算进了当下的公式里。它求稳，最终训练出来的策略是：宁可多扣点分，也绝对不走悬崖边。它会选择**【走安全内道】**。（这叫 On-policy，保守、安全）

键盘侠 Q-learning 的抉择：当 Q-learning 评估“走悬崖边缘”时，它完全不考虑自己等会儿会不会作死。它心想：“只要我走在悬崖边，只要下一秒我不乱选、保持神级操作，我的潜力

就是无限的 ($\max Q$)!” 它用 “最完美的未来假设” 更新了现在。于是它得出结论：走悬崖边价值最高！但讽刺的是，在实际游戏时，因为它有 10% 的探索概率，它走着走着还是会手贱失足摔死。但它哪怕摔死了，在写报告（更新公式）时，依然坚持认为走悬崖边是最好的。最终，Q-learning 训练出来的最优路径是：大摇大摆地【走悬崖边缘】。（这叫 Off-policy，激进、激进、贪婪）

表 2: 时序差分算法流派对比：SARSA vs Q-learning

对比维度	SARSA 算法	Q-learning 算法
核心属性	同策流派 (On-policy)	异策流派 (Off-policy)
哲学人设	胆小求稳的“老实人”	理想主义的“键盘侠”
更新依据	依据”实际执行”的下一步动作 A'	依据”理想中完美”的动作最大潜力 $\max Q$
关键五元组	严格依赖 (S, A, R, S', A') 五张牌	只依赖 (S, A, R, S') , 无需实际的 A'
TD 目标值	$R + \gamma Q(S', A')$	$R + \gamma \max_{a'} Q(S', a')$
训练风格	保守，更新时会将自身的随机探索风险（如手贱作死）考虑在内	激进，更新时假设未来每一步都完美无瑕，不考虑自身探索失误
独木桥抉择	宁可多扣点分，也绝对不走容易脚滑的悬崖边（选择安全内道）	只要不脚滑的潜力最大，就大摇大摆走悬崖边（选择极限路径）
应用场景	适合在线学习、真实物理系统（容错率低、防止拆机）	适合仿真环境、离线数据训练、游戏智能体（追求极限高分）

9、DQN 算法

在 Q-learning /SARSA 法中，我们以矩阵的方式建立了一张存储每个状态下所有动作值 Q 的表格。表格中的每一个动作价值 $Q(s, a)$ 表示在状态 s 下选择动作 a 然后继续遵循某一策略预期能够得到的期望回报。然而，这种用表格存储动作价值的做法只在环境的状态和动作都是离散的，并且空间都比较小的情况下适用，当状态或者动作数量非常大的时候，这种做法就不适用了。

为了解决这个“格子不够用”的问题，研究者们提出了一个伟大的构想：我们能不能不要表格了？用一个“超级函数”来代替表格。在 DQN 中，我们把这张笨重的 Q 表格扔掉，换成了一个深度神经网络 (Deep Neural Network, DNN)，这个网络在算法里被称为 Q 网络 (Q-network)。

在 Q-learning 中，我们查表的过程是：输入：当前状态 $s \rightarrow$ 动作： $a \rightarrow$ 翻箱倒柜查表 $\rightarrow$ 输出： $Q(s, a)$ 的数值。

在 DQN 中，输入：当前状态的连续数值 s （比如输入一个向量：[位置, 速度, 角度]） $\rightarrow$ 通过多层神经元计算 $\rightarrow$ 输出：所有动作的 Q 值（比如输出一个向量：[爬的 Q 值, 跑的 Q 值]）。

此时，神经网络变成了一个“动作价值近似器 (Function Approximator)”。它的最大好处是：它具有超强的“举一反三（泛化）”能力。哪怕智能体从来没有经历过位置 =1.001 米，速

度 = 2.003 米/秒这种极其精确的状态，神经网络也能根据之前在类似状态下的训练经验，极其聪明地“估算”出此时【跑】和【爬】的 Q 值。

9.1 DQN 的损失函数

我们知道，训练神经网络需要损失函数 (Loss Function)，然后通过梯度下降 (Gradient Descent) 去更新网络的权重参数 θ 。DQN 再次展现了时序差分 (TD) 的纯正血统，它把 TD 误差直接做成了损失函数！

$$L(\theta) = \mathbb{E} \left[\left(\underbrace{R + \gamma \max_{a'} Q(S', a'; \theta^-)}_{\text{TD 目标值 (标签)}} - \underbrace{Q(S, A; \theta)}_{\text{当前网络预测值}} \right)^2 \right]$$

你仔细看这个公式，它和 Q-learning 的大骨架完全是一模一样的：当前网络预测值：网络输入当前状态 S ，输出动作 A 的当前 Q 值。TD 目标值（相当于监督学习里的标签 Label）：利用大自然给的即时回报 R ，加上网络对下一步状态 S' 的最优潜力预测 $\max Q$ 。计算误差：用“理想的潜力（目标值）”减去“当下的预测”，算一个均方误差 (MSE)。反向传播：误差越大，说明网络猜得越不准，通过反向传播调整网络权重 θ ，让网络下一次猜得更准。

如果直接把 Q-learning 的公式套进神经网络，网络大概率会直接崩掉（无法收敛）。因为深度学习有一个致命的前提：训练数据必须满足独立同分布 (i.i.d)。而强化学习的数据是智能体在环境里一步步走出来的，前后两步高度相关（时间序列相关），这会导致网络产生严重的过拟合和震荡。为了让 DQN 真正能用，DeepMind 团队提出了两个里程碑式的发明（这也是写进教科书的重点）：

1. 经验回放 (Experience Replay) ——打破数据的因果死锁智能体把每次肉身试错的五元组 $(S, A, R, S', \text{done})$ 不要立刻拿去喂给网络，而是先扔进一个巨大的“记忆库 (Replay Buffer)”里。当网络要学习时，从这个记忆库里随机抽取 (Random Sample) 一个批量 (Batch) 的数据来训练。物理含义：这就像打破了时间的连续性，把记忆打碎了随机回想，让网络在“温故而知新”中摆脱了前后动作的死连带关系。

2. 目标网络 (Target Network) ——解决“追逐自己影子”的震荡如果你看上面的损失函数公式，你会发现更新权重 θ 的时候，由于目标值里也包含 $Q(S', a'; \theta)$ ，这导致“你一边修改网络的参数，你的训练标签 (Label) 同时也在跟着动”。这在工程上就像狗追自己的尾巴，极难收敛。DQN 的解决办法是：手里准备两个一模一样的网络。一个叫当前网络（负责天天被训练、天天改参数 θ ），另一个叫目标网络（参数 θ^- 固定不动，专门用来提供稳定的标签）。每隔 1000 步，才把当前网络的参数复制给目标网络一次。

10、从动态规划到无模型学习的演变

表 3: 从动态规划到深度强化学习 (DRL) 的算法演进全景表

算法 / 流派	核心致命痛点(面临的问题)	演进的核心动机 (为什么这么改)
动态规划 (DP) (策略/价值迭代)	依赖完美模型 (上帝视角) 现实中大自然的转移概率 $p(s' s, a)$ 和回报函数 $r(s, a)$ 极其复杂, 根本无法建出精确的数学模型。	走向无模型 (Model-free) 扔掉环境说明书, 驱使智能体亲自去环境中单线采样、肉身试错, 用真实发生的经验代替理论推导。
蒙特卡洛 (MC)	秋后算账、效率极低 必须一路走到黑、直到游戏彻底结束拿到总分 G_t 才能更新。若任务没有明确终点或序列极长, 算法直接瘫痪。	引入时序差分 (TD) 机制 走一步看一步, 利用“即时回报 $R +$ 下一步的预估价值 $V(S')$ ”进行自举更新, 无需等待通关。
SARSA	过于保守、无法利用离线数据 属于同策流派 (On-policy), 更新时必须捆绑下一步实际执行的动作 A' 。如果探索时有作死倾向, 会将当下的动作价值大幅调低。	解耦评估与执行 (异策化) 演进出 Q-learning 。更新时直接取下一步潜力的最大值 $\max_{a'} Q(S', a')$, 用理想最优路径来更新现在, 表现更具攻击性。
Q-learning	维数灾难、无法处理连续空间 依赖笨重的表格 (Q-table) 存储数据。当面对无人机、机器人等连续、无限状态空间时, 表格尺寸爆炸, 内存直接崩溃。	引入深度神经网络 (函数逼近) 演进出 DQN 。用多层神经元(Q网络)代替表格, 输入连续状态直接输出动作价值, 具备强大的“举一反三”泛化能力。
基础 DQN	数据关联性强、标签不稳定 强化学习数据前后高度相关, 违反深度学习独立同分布前提; 且更新时“狗追尾巴”, 目标值随参数实时震荡, 极难收敛。	经验回放与目标网络 (完全体) 引入 Replay Buffer 打碎时间相关性; 引入双网络机制锁死 Target 标签, 让深度强化学习在工程上真正稳定落地。

11、总结

本文仔细分析了从动态规划到无模型学习的发展历程，解决了提及算法 What, why, how 的问题。但无论上述哪种算法，无非都是基于状态价值 V 或者动作价值 Q 的方法，本质上是基于价值的。在接下来的学习中，我将学习到基于随机策略的方法（使用神经网络的策略梯度，自然策略梯度，信任区域策略优化（TRPO），近端策略优化（PPO），A3C）和基于确定性策略的方法（确定性策略梯度（DPG），DDPG）。

12、Cartpole 实验仿真

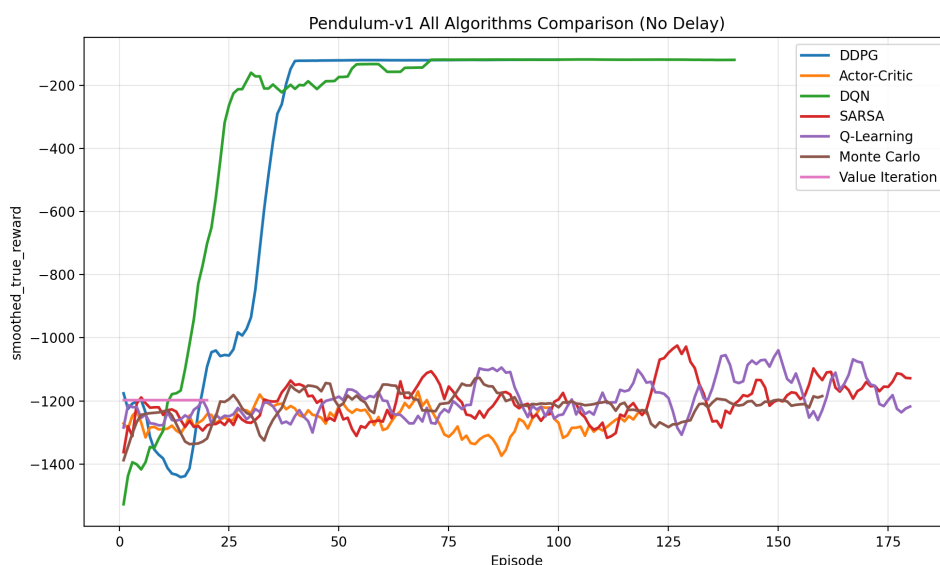


图 3: 不同强化学习算法在倒立摆环境下的训练回报对比

为了比较不同强化学习算法在倒立摆控制任务中的性能，本文在无奖励延迟条件下，将 Value Iteration、Monte Carlo、SARSA、Q-learning、DQN、DDPG 以及 Actor-Critic 的训练结果绘制在同一张图中，并以平滑真实回报（smoothed true reward）作为主要评价指标。由于该环境的 reward 越接近 0 表示控制效果越好，因此曲线越高，说明算法性能越优。

从图中 3 可以看出，不同算法之间存在明显性能差异。DDPG 与 DQN 的曲线整体最高，并且在训练后期稳定在约 -120 附近，说明这两种方法能够较好地学习到倒立摆的稳定控制策略。其中，DDPG 在连续动作控制任务中表现尤为突出，这与其面向连续动作空间设计的特点一致；DQN 虽然作用于离散化后的动作空间，但仍取得了接近 DDPG 的效果，说明其值函数逼近能力较强。

相比之下，SARSA、Q-learning、Monte Carlo 和 Value Iteration 的表现明显较弱，其最终回报大多仍停留在 -1000 左右。这表明在离散化倒立摆环境下，传统表格型方法虽然能够在一定程度上改善控制性能，但受限于状态和动作离散化精度，难以逼近连续控制下的最优策略。尤其是 Value Iteration，虽然在理论上可以求得最优策略，但这里基于离散近似模型进行求解，因此最终性能受到较大限制。

此外，Actor-Critic 在本实验中的表现也不理想，曲线明显低于 DDPG 和 DQN，且波动较大。这说明基础 Actor-Critic 在该任务上训练稳定性不足，策略更新容易受到高方差估计的影响。

综合来看，本实验可以得到以下结论：(1) 在倒立摆控制任务中，深度强化学习方法整体优于传统表格型方法；(2) DDPG 在连续控制问题上最具优势；(3) DQN 在离散化设定下表现最好，是离散动作方法中的最优者；(4) Monte Carlo、SARSA、Q-learning 和 Value Iteration 更适合作为对比基线，能够说明离散化和经典算法在连续控制任务中的局限性。

13、疑问与解答

Q1: 但我不太理解，在序列路径中，选择不同节点已经有了环境转移概率，这和策略概率是一个东西吗，或者说上述概率已经隐含了策略？

A1: 它们不是同一个东西，但它们共同决定了序列路径的概率。你之所以觉得路径中选择不同节点已经有了概率，是因为在现实中，我们看到的是一个最终融合出来的结果。但在强化学习的数学框架里，这个概率被严谨地拆分成了两个独立演化的力量：主观力量：策略 $\pi(a | s)$ ——智能体想怎么选（由智能体控制）。客观力量：环境转移概率 $p(s' | s, a)$ ——环境会怎么变（由大自然或游戏规则控制）。我们用你要求的“单向、两分支序列”把这两个概率彻底剥离来看。

Q2: 终点的价值是多少？

A2: 在终点，因为没有后续节点，它的未来回报期望为 0，所以该点的价值 (Value) 在数值上正好就等于它受到的即时奖励 (Reward)。